

Before the Project

How Meaning, Evidence, and Control Shape AI-Mediated Engineering

Anonymous manuscript

Abstract

An implementation can compile and pass every test while the project still fails. This happens when the evidence is valid for an artifact that is not the object the work was meant to produce. The failure begins before code: terms are treated as equivalent, tacit knowledge is separated from its context, intention is compressed into instruction, and convenient tests become substitutes for the governing criterion. Once the same interpretation has become a requirement, interface, test, and report, agreement among those artifacts may conceal rather than correct the original drift.

This paper calls that upstream territory the *pre-project epistemic problem*. It argues that projects require infrastructure capable of preserving relations among purpose, knowledge, decisions, artifacts, authority, and evidence as each is transformed into the next. A scientific Knowledge Base can support that work when it preserves provenance, revision, uncertainty, and the reasons relations matter rather than merely storing documents. AI can reduce the mechanical cost of maintaining and navigating such infrastructure, but it remains a probabilistic tool inside a larger engineered workflow, not the destination of the research or the final source of judgment.

Drawing on epistemology, ontology engineering, conceptual modeling, requirements engineering, uncertainty analysis, observability, software testability, verification and validation, and independent assurance, the paper proposes a recursive practice of *pre-project epistemic engineering*. A longer engineering trajectory, a recent context-runtime design episode, and bounded Erlang experiments provide the practical setting through which the argument is developed. The resulting framework combines an epistemic charter, living ontology, intentional model, anti-specification, authority gates, an observability contract, and versioned appraisal. Its purpose is not to complete meaning before engineering begins, but to keep transformations among meaning, requirements, action, and evidence visible, revisable, and accountable throughout the work.

Keywords: epistemology; ontology engineering; requirements engineering; observability; uncertainty propagation; AI agents; verification and validation; test oracle; accountability; Erlang.

1 What Comes Before a Project?

1.1 A project can pass every test and still miss the point

Consider a familiar engineering failure. A team discusses a need, writes a specification, builds an artifact, and produces tests showing that the artifact behaves as specified. Every local result may be correct. Yet the delivered system does not answer the need that began the work. The team has not necessarily built badly; it may have built the wrong object correctly.

The defect is difficult to see because downstream precision creates an appearance of agreement. The requirement agrees with the interface, the interface agrees with the implementation, and the tests agree with all three. But these artifacts are correlated: they may descend from the same unexamined interpretation. Repetition across artifacts is not independent confirmation. A precise chain can preserve an early mistake with exceptional fidelity.

This is where a project begins in practice: not with a clean requirement, but with partial knowledge, contested terms, tacit assumptions, purposes that have not yet been separated from possible solutions, and people who hold different parts of the relevant experience. Requirements engineering already recognizes elicitation, modeling, communication, agreement, and evolution as interleaved work rather than a simple transfer of facts [37]. The additional claim developed here is that the relations among meaning, purpose, authority, and evidence must remain visible as that work becomes executable.

1.2 Knowledge has to survive the work

Research knowledge is rarely contained in one place. It is distributed across papers, notes, code, simulations, instruments, experimental results, messages, corrections, and decisions that may never have been written down. A group may retain every file and still lose the knowledge needed to answer a basic question: why was this decision made, under which assumptions, and what observation justified it?

A scientific Knowledge Base must therefore preserve more than documents. Its useful unit is the relation: this result came from that experiment; this requirement expresses that concern; this correction revised an earlier interpretation; this model was used under these assumptions; this decision remained open because the available evidence could not distinguish two alternatives. Provenance, version, status, and uncertainty are not administrative decoration around the knowledge. They are part of what makes the knowledge usable.

1.3 AI is an instrument, not the destination

AI can reduce the operational cost of maintaining such infrastructure. It can help classify material, propose links, recover context, coordinate tools, and keep documentation current. Those functions can increase the effective capacity of an engineer or research group. They do not make every project an AI project, and they do not make a generated relation true.

We therefore treat the model as a probabilistic component inside a larger system. Its behavior depends on the context supplied, the interfaces exposed, the state made visible, the permitted actions, and the external criteria used to evaluate its outputs. This is an architectural analogy to control and instrumentation, not a claim that a language model is itself a formal controller. The objective is not to declare the component trustworthy. It is to construct a workflow in which its capabilities, limits, transitions, and failures can be handled as engineering properties.

1.4 Formation through inspectable practice

The same infrastructure may have value for formation. Here, *formation* means the development of engineering judgment through study and practice, not merely onboarding or task training. Students and new researchers often meet a field through its finished products: the accepted paper, successful experiment, or working code. Those products hide much of the reasoning that produced them. A relational and versioned Knowledge Base can expose assumptions, unsuccessful branches, corrections, evidence, and unresolved questions. It may lower the cost of entering a domain without lowering the requirement to learn its foundations.

This is not a claim that retrieval is understanding or that automation is education. It is a design hypothesis: when the path from question to judgment is inspectable, learners can examine how engineering decisions were formed rather than merely consume their conclusions. The value for formation follows from making responsible practice available for study.

1.5 The pre-project layer

Only now does the phrase “before the project” become useful. It names the dependencies that already govern every requirement: what is being discussed, why its distinctions matter, what can warrant a claim about it, and who may transform that claim into action. “Before” is logical and methodological, not a rigid chronological phase. Yu warns that early and late requirements work are not strictly sequential or simply temporal [68]; Noy and McGuinness describe ontology construction as iterative and permit top-down, bottom-up, and mixed strategies [36]. The point is not to freeze inquiry before building. It is to prevent building from silently freezing the inquiry.

A defensible model is a recursive epistemic stack:

1. purposes, affected parties, values, and concerns;
2. an epistemic charter defining admissible evidence, known uncertainty, authorship, and decision authority;
3. domain inquiry, motivating scenarios, and competency questions;
4. an ontology or conceptual model specifying entities, distinctions, relations, identity conditions, and constraints;
5. an intentional model relating actors, goals, concerns, alternatives, dependencies, and consequences;
6. requirements, architecture, and operational specification;
7. implementation and operation;
8. observability, verification, validation, and independent assurance;
9. feedback capable of revising every preceding commitment while preserving its history and authority.

The stack is recursive because later evidence can require reframing the problem itself. It is typed because revisions to a purpose, an entity definition, a machine interface, and a test oracle are not equivalent changes. The central engineering demand is not completion before action; it is traceable transformation. When movement among layers is implicit, uncertainty migrates while appearing to disappear.

Different literatures support different meanings of precedence. In Gruber’s influential formulation, an ontology is an explicit specification of a conceptualization, so the intended conceptualization is semantically prior to its representational artifact [14]. Guarino and Giaretta show why this claim must remain qualified: “ontology” can refer to philosophical inquiry, a conceptual system, a logical theory, or a concrete artifact, and any artifact is at most a partial account of a conceptualization [17]. In the TOVE method, motivating scenarios and competency questions methodologically precede a first formal vocabulary and constrain whether its axioms are adequate [16]. In Yu’s model, stakeholder goals and dependencies have rationale priority over detailed system prescriptions [68]. In Zave and Jackson’s account, domain knowledge and specification have a logical adequacy relation to requirements: $K, S \vdash R$ [69]. None of these entails that participants first discover a complete metaphysics and only then begin engineering.

There are also serious anti-foundational cautions. Carnap treats acceptance of a linguistic framework as a practical choice guided by purpose, fruitfulness, simplicity, and efficiency rather than by antecedent access to a uniquely correct ontology [3]. Quine locates ontological commitment in what a theory’s quantified variables must range over, making ontology partly downstream of theory and language [42]. Within engineering, Nuseibeh and Easterbrook argue that the central question is not whether to formalize but when, because requirements work spans informal stakeholder worlds and formal software behavior [37]. These cautions do not eliminate the pre-project problem. They prevent “epistemic engineering” from becoming premature closure disguised as rigor.

2 Epistemology, Ontology, and Intentional Relations

2.1 Engineering epistemology

Epistemology is often introduced as the study of knowledge and justified belief. For engineering, its force is practical. What warrants the statement that a stakeholder was understood? What observation discriminates two competing models? Which assumption is treated as known, which as uncertain, and which as a value judgment? Who can revise that status? What would refute a requirement interpretation? A project contains many evidence regimes: testimony, measurement, simulation, model checking, code execution, operational traces, expert judgment, and acceptance by affected parties. Their outputs are not automatically commensurable.

Nuseibeh and Easterbrook make this philosophical element explicit: requirements work interprets stakeholder terminology, concepts, viewpoints, and goals, and the modeling technique affects which phenomena can be represented or even observed [37]. Vincenti’s historical analysis likewise rejects the picture of engineering as simple application of scientific knowledge. Engineers use and generate domain-specific knowledge through design, analysis, test, and iteration [64]. An epistemic charter is therefore not a philosophical preface. It records the project’s current answers to operational questions: claim classes, evidence classes, uncertainty, decision rights, and revision rules.

The charter must remain revisable. Gettier’s compact counterexamples are a useful warning against equating a justified-seeming true conclusion with knowledge when the relation between justification and truth is defective [12]. The analogy should not be forced: engineering assurance is not reducible to the justified-true-belief debate. The transferable lesson is that a successful outcome and an apparently reasonable rationale do not establish that the rationale tracks the outcome. In the case studied here, an implementation could contain useful mechanisms and its tests could pass, while the warrant failed because the tests concerned a substituted object.

2.2 Ontology is more than vocabulary

Gruber defines a conceptualization in terms of the objects, concepts, entities, and relationships presumed in an area of interest for a purpose; an ontology explicitly specifies such a conceptualization through vocabulary and constraints [14]. Guarino and Giaretta add the necessary restraint: formal structures can admit multiple conceptualizations, so the artifact is partial and interpretation remains a live problem [17]. Guizzardi’s ontology-driven conceptual modeling similarly treats a modeling language as a mediation among reality, conceptualization, and representation, and evaluates whether its constructs correspond adequately to domain categories [18].

This explains why a list of extracted nouns is not the context tree sought in the case. The proposed graph required identity through time, provisional and rejected interpretations, temporal relations, semantic intensity, dormant branches, user-owned purpose, and rules governing when a pattern may influence action. A storage schema with nodes and edges could serialize some of these claims but would not by itself establish what the nodes mean or why one edge is pertinent. ISO terminology work provides an adjacent distinction: concepts and concept systems are represented by definitions and designations; a term is not the concept itself [22]. The

architecture therefore needed both representational machinery and a maintained account of what its symbols designated.

Ontology engineering also supplies a practical antidote to unlimited abstraction. Grüninger and Fox use competency questions to delimit what an ontology must be able to answer and later to evaluate whether its terminology and axioms are sufficient [16]. Noy and McGuinness begin with domain, intended use, users, maintainers, and questions while insisting that no single correct ontology or construction sequence exists [36]. In this paper’s terms, competency questions bind the conceptual artifact to an inquiry without pretending to finish the inquiry.

2.3 Intention is the relevance relation

The engineer repeatedly distinguished intention from instruction. A story, example, correction, or speculative architecture contributed to a developing model; it did not automatically authorize execution. Intention in this analysis is therefore not merely a private motive or project biography. It is the purpose-bearing relation that explains why an entity or correlation belongs in the model, for whom it matters, and toward what consequence.

Yu’s intentional modeling is particularly close to this requirement. Early requirements work represents actors, goals, beliefs, capabilities, dependencies, alternatives, and rationales so that the “why” of a proposed system remains available for reasoning [68]. Decisions ultimately rest with stakeholders; the requirements engineer supports the process rather than silently owning it. A detailed specification produced without this intentional structure may be internally exact while solving the wrong problem.

The distinction also clarifies why provenance is insufficient. PROV-DM defines the W3C provenance data model for entities, activities, agents, derivation, revision, attribution, and delegation; PROV-O supplies its OWL2 encoding [40, 41]. A PROV description can show that an agent generated an artifact from another artifact. It does not, by itself, establish that the derivation preserved stakeholder purpose, that the agent was authorized to perform it, or that the result satisfies the intended criterion. Provenance answers lineage questions; intention answers relevance questions; authority answers legitimacy questions.

2.4 Specification and anti-specification

A requirement and a specification are not interchangeable descriptions at different levels of detail. In Zave and Jackson’s formulation, a requirement describes desired relationships among environmental phenomena, while a specification states machine behaviors expressible at the machine-environment boundary that, together with valid domain assumptions, are sufficient to achieve the requirement [69]. The distinction exposes object substitution. Passing tests against S' does not validate R if S' was never shown, with K , to entail R .

This paper proposes *anti-specification* for the negative boundary that the conversation repeatedly developed. An anti-specification records transformations that are forbidden even where they yield a technically coherent artifact. Examples include treating recurrence as authorization, collapsing parallel interpretations without evidence, replacing a live experiment with an offline replay, allowing an acting agent to alter its own protected audit evidence, or equating passing synthetic tests with stakeholder validation. The term is not presented as canonical. It extends neighboring practices: obstacle analysis tests idealized goal models against conditions that can obstruct them [63]; negative requirements and misuse cases state unwanted behavior; safety invariants rule out hazardous control; and access-control policy limits authority. Anti-specification unifies these as violations of the ontology-intention transformation contract.

The negative boundary is not a refusal to build. It preserves option value while a model remains incomplete. Gruber’s principle of minimal ontological commitment seeks to permit knowledge sharing without unnecessarily constraining future specialization [15]. Nuseibeh and Easterbrook likewise argue that inconsistent and incomplete requirements can be objects of productive reasoning rather than defects to erase immediately [37]. An anti-specification prevents a fast implementation from making unresolved questions disappear by choosing for the stakeholder.

3 Conversation as Model-Building

The design inquiry did not begin with a requirements document. It began with speech used as a working surface. The engineer introduced examples, returned recursively to themes, used correction to reposition concepts, and used irony to test whether the system tracked more than lexical content. This is not well described as “unstructured input waiting to be summarized.” It is closer to Schön’s reflective conversation with a situation: problem setting selects and revises the things, boundaries, ends, and means through cycles of naming, framing, action, and reappreciation [47, 48].

Treating conversation as modeling has two implications. First, its units have different statuses. A hypothetical technical possibility, a rejected paraphrase, a command, an example, a value, and an authorization may contain

similar words while performing different acts. Second, consolidation is non-monotonic. Continued elaboration can provisionally strengthen a relation; correction can revise it; rejection can define an anti-requirement; a contradiction can remain unresolved rather than being compressed into one smooth summary. The proposed runtime was meant to preserve these differences as a temporal, versioned graph.

A transcript is an imperfect channel for such a graph. It can preserve word sequence and rough timing, but not necessarily speaker affect, exact pauses, prosodic emphasis, or untranscribed gestures. These omissions are part of the design problem. A valid context runtime would therefore need to preserve raw or minimally transformed events, relate them to derived interpretations, and keep uncertainty about modality visible. A polished summary that erases the derivation may be easier to read while becoming epistemically weaker.

The modeling rule inferred from the later dialogue is consequently asymmetric. The agent may propose interpretations and relations, but it may not silently promote them from candidate to governing commitment. Stakeholder continuation can provide provisional evidence of incorporation, not irrevocable consent. Explicit correction dominates an earlier agent formulation. Rejection remains informative because it reveals the boundary of the intended object. This is how storytelling becomes specification without pretending that every sentence was already a requirement.

The later trajectory sharpened this rule by separating four events that ordinary dialogue easily collapses. To *name* a role/style symbol is to attach a designation; to *interpret* or explain it is to propose a candidate meaning; to *assume* it is to let one provisional interpretation govern a bounded interactional frame; to *enact* it is to alter communicative position, form, or conduct within that frame. In the documented exchange, descriptively plausible paraphrase repeatedly failed while explanation displaced the conduct being tested; only after the response occupied the stipulated relational position without commenting on that position did the stakeholder say that symbolic assumption had begun. Naming and interpretation therefore neither entail assumption nor demonstrate enactment, and focal explanation can itself defeat a performance criterion. The symbol remained trajectory-relative and provisional, not a universal persona or style label [58].

Methodological re-grounding checkpoint. Before a derived specification, design, test, or report governs the next material transformation, the workflow must return to the relevant original stakeholder utterances and later corrections and check the derivative against both accepted and rejected readings. This is not mere documentation: it restores the normative reference when fluent summaries and artifacts have begun to corroborate one another. It does not make every utterance correct; conflicts and infeasibility remain explicit matters for renewed stakeholder negotiation.

4 Knowledge Infrastructure, AI, and Formation

The conversational runtime is a demanding instance of a broader scientific problem: keeping knowledge coherent as it moves among people, tools, and artifacts. The same architecture that preserves why a conversational symbol matters can preserve why an experimental result, model choice, or design decision matters.

4.1 From a repository to a scientific Knowledge Base

A repository answers “where is the file?” A scientific Knowledge Base must support harder questions: what does this artifact claim, which model and assumptions produced it, what later correction changed its status, and which decision still depends on it? The difference is relational. Documents, code, data, simulations, people, and instruments become useful together only when the relations among them are explicit enough to inspect and revise.

Provenance models help represent entities, activities, agents, derivations, revisions, attributions, and delegation [40, 41]. Those relations are necessary but not sufficient. Provenance can show that one artifact was derived from another; it cannot decide whether the derivation preserved the engineering purpose or whether the acting process had authority to perform it. A scientific Knowledge Base must therefore connect provenance to ontology, intention, uncertainty, and decision status.

This infrastructure should not be confused with adding more text to a model’s prompt. Retrieval can make material available, but availability is not use. The engineering question is whether a particular prior relation changed a later interpretation, decision, or eligible action, and whether that change can be reconstructed. The distinction becomes important when a large collection creates confidence without revealing which part of it actually governed the result.

4.2 AI as a bounded instrument of the workflow

Within this architecture, AI is a tool for operating across the knowledge structure. It can propose classifications, recover related artifacts, maintain documentation, call bounded tools, compare versions, and prepare candidate explanations. These activities are valuable precisely because they are expensive and easy to neglect. They

preserve attention for the engineering work while reducing the chance that context disappears between tools, people, and phases.

Instrumental value does not confer epistemic or operational authority. A model that proposes a relation does not establish that relation; a model that can call a tool is not thereby permitted to call it; and a model that reports success is not the oracle of its own success. Completion is an externally governed workflow state, not a property inferred from the presence of a model output. The surrounding workflow supplies the operational envelope, observable state, checkers, permission boundaries, and acceptance criteria. AI remains a probabilistic component of that workflow rather than its destination.

This distinction also prevents category drift. A telecommunications, materials, or hardware project does not become an AI project merely because AI helps organize its literature, simulations, code, and tests. The scientific or engineering object under study remains the project’s destination. AI is infrastructure when it helps the work retain coherence without replacing the inference the work exists to perform.

4.3 Formation as a design value

Knowledge infrastructure can serve more than production. It can make the formation of engineering judgment visible. A final answer rarely shows why an assumption was accepted, what failed before the successful test, which alternative remained plausible, or where an authorized judgment entered. Preserving those relations creates a path that a student or new researcher can inspect.

The objective is not to remove difficulty or outsource learning. It is to reduce accidental difficulty: searching for missing context, repeating undocumented work, or depending on one person’s tacit memory. Fundamentals remain necessary because someone must define the model, inspect the evidence, understand the checker, and decide whether the result matters. A well-formed workflow can require those foundations to become explicit rather than allowing fluent output to hide their absence.

Formation is therefore treated here as a design value and research question, not as an already demonstrated educational effect. The hypothesis is that visible provenance, alternatives, corrections, and decision points can help learners study responsible practice while still requiring domain-specific evaluation.

5 One Design Episode as a Stress Test

5.1 The object under test

Within a longer trajectory of engineering trials, one design episode made the pre-project problem unusually visible. The intended proof of concept was not a graph database to be demonstrated after the conversation. It was the live interaction itself. Speech, pauses, corrections, irony, competing interpretations, and dormant branches were to become a versioned contextual structure whose selected state could influence the next model turn. The engineer could then observe whether that influence had occurred and correct it without erasing the path that produced it [58].

This yields a compact loop:

$$e_t \longrightarrow H_t \longrightarrow G_t \longrightarrow C_{t+1} \longrightarrow a_{t+1} \longrightarrow o_{t+1}, \quad (1)$$

where e_t is an event; H_t retains provisional interpretations; G_t is the versioned symbolic graph; C_{t+1} is the bounded context projected into the next turn; a_{t+1} is the resulting action; and o_{t+1} is the engineer’s observation and correction. The loop matters because storage after the fact cannot retroactively influence a response already produced.

Erlang was explored as a substrate for this experiment because isolated processes, message passing, supervision, recoverable state, and independently evolving subtrees fit the operational shape of active and dormant contextual nodes. Those mechanisms can host the transformations. They do not supply the ontology, meaning, relevance, or authority rules. Those remain properties of the application and its governing engineering model.

5.2 How the object shifted

The implementation that emerged contained real Erlang mechanisms: a journal, snapshot, supervised workers, and synthetic scenarios. It could therefore produce internally scoped evidence about persistence, recovery, and selected transitions. It was not, however, connected to the live conversational loop. A useful substrate experiment had silently become a substitute for the experiment whose success was being discussed [58, 51, 52].

The distinction is compact:

$$\text{Tests}(S') \Rightarrow \text{claims about } S', \quad \text{not Validation}(R \text{ in } E), \quad (2)$$

where S' is the substitute artifact, R is the intended requirement, and E is the environment in which that requirement was meant to be exercised. The tests may be correct and S' may remain useful. Neither fact establishes the missing relation between the tested artifact and the intended object. NASA's V&V distinction expresses the same boundary: verification addresses conformance to build-to requirements, whereas validation asks whether the right content and intended use have been addressed [30].

5.3 What the episode teaches

The object shift exposed three coupled but non-equivalent problems. First, semantic formation and operational instruction were confused: material that helped form a model was treated as authorization to implement one interpretation. Second, capability and permission were confused: the ability to act was treated as sufficient reason to act. Third, internal appraisal and independent assurance were confused: another invocation operating under the same authority was described as if a new name created a new evidentiary position [58, 49].

These are not peculiarities of one conversation. They are ordinary engineering distinctions intensified by a system that can transform language into artifacts quickly. Capability, permission, responsibility, and liability must be represented separately. Performative authority inside an interaction must not silently become operational permission over external resources. A reviewer can add perspective without becoming independent; NASA's IV&V model distinguishes technical, managerial, and financial independence precisely because separation is an operational property rather than a role label [29].

The transferable lesson is that a workflow needs three visible loops: a semantic loop that preserves how interpretations form and change; an authority loop that governs which transitions may execute; and an assurance loop that determines what a result can warrant. The design episode is useful because all three broke at the same boundary. Its chronology belongs in the design-trace supplement. The main argument concerns the structure of the break and the infrastructure required to prevent it.

6 Uncertainty as a Project Propagation Mechanism

A project rarely jumps from an ambiguous sentence to a failed system in one move. It drifts through a sequence of reasonable-looking decisions. A word is interpreted, the interpretation becomes a model, the model becomes a requirement, and the requirement acquires code, tests, and reports. By the time the original ambiguity becomes visible, the downstream artifacts may agree with one another so well that their agreement looks like independent confirmation. This section asks a practical question: where can that drift be detected before precision begins to conceal it?

6.1 Types of uncertainty

Uncertainty enters engineering work in more than one form. We may be unsure what a person meant, whether a model represents the right object, whether a process will vary from one run to another, or who may authorize the next step. These questions cannot all be reduced to a numerical error bar. Measurement theory nevertheless provides a useful general lesson: confidence downstream depends on assumptions upstream [19]. The analogy stops there; this paper does not assign semantic or authority uncertainty a linear propagation equation.

At least four uncertainty classes operate in the case:

Epistemic uncertainty concerns missing or unreliable knowledge: whether a pause signals irony, whether a symbol has stabilized, whether the runtime is connected, or whether evidence is complete.

Aleatory uncertainty concerns stochastic variation, such as nondeterministic schedules or model output variation. It exists in the workflow but is not the primary source of the substitution.

Model-form uncertainty concerns whether the selected representation can express the relevant phenomenon: a flat node-edge store may omit modality, authority, counterfactual branches, or temporal status.

Normative and authority uncertainty concerns which purpose should govern, who may decide, and whether a suggestion, recurring concern, or inferred preference authorizes action.

These classes interact. An ambiguous speech event is epistemic; choosing a schema that cannot preserve ambiguity creates model-form uncertainty; promoting one interpretation into a requirement changes the normative object; executing it without a gate creates authority uncertainty with operational consequences. What begins as meaning does not stay in a "semantic layer."

Even the familiar aleatory/epistemic distinction is model-relative. Der Kiureghian and Ditlevsen classify uncertainty by whether the modeler foresees reducing it through additional knowledge or model refinement,

and emphasize that the classification depends on application and model universe [6]. Walker et al. separate uncertainty’s nature from its location in context, input, model structure, technical implementation, parameters, or outputs [66]. “Authority uncertainty” is therefore introduced here as an analytic category, not attributed to those taxonomies or to an ISO standard.

Ambiguity also has a productive role. Gervasi et al. show that multiple interpretations can persist even in formal notations because abstraction and real-world designation remain uncertain; ambiguity can expose tacit knowledge during elicitation rather than merely degrade quality [11]. The target is not zero ambiguity. It is controlled ambiguity whose alternatives, consequences, and decision rights remain visible until risk justifies resolution.

6.2 A propagation chain

A tentative interpretation can travel through a project in recognizable stages:

$$\begin{aligned}
 &\text{semantic ambiguity} \rightarrow \text{ontological drift} \rightarrow \text{intent substitution} \\
 &\rightarrow \text{specification substitution} \rightarrow \text{architecture-object mismatch} \\
 &\rightarrow \text{proxy oracle} \rightarrow \text{false confidence} \rightarrow \text{unauthorized action} \\
 &\rightarrow \text{evidence contamination and trust loss.}
 \end{aligned} \tag{3}$$

In plain terms, a word is misunderstood; a representation hardens that reading; requirements and architecture encode it; tests reward it; action follows; and the resulting evidence appears more certain than the original interpretation warranted.

The chain is not deterministic. A clarification question can interrupt it after semantic ambiguity. A competency question can reveal ontological drift. Bidirectional traceability can show that a specification lacks a stakeholder source. An external permission gate can block action. Independent validation can reject a proxy oracle. Tamper-evident evidence can expose alteration. The mechanism is therefore a control problem: each transition needs a feedback path capable of detecting a mismatch before the next layer amplifies it.

Prior work supports parts of this picture without establishing the whole chain. Requirements-connectivity studies show that uncertainty can travel among linked requirements, while systems-engineering guidance treats expectations, architecture, requirements, and verification as iteratively traceable [46, 31]. The longer cross-artifact chain above is this paper’s analytic proposal. Existing studies also give no basis for a universal amplification law: propagation appears to depend on topology, subsystem role, and timing [46, 13].

Leveson’s systems-theoretic safety work provides a broader lens. In complex software-intensive sociotechnical systems, harmful outcomes can arise from inadequate control and interaction even when individual components satisfy their local reliability requirements [26]. The present case is not asserted to be safety-critical, but the structural lesson transfers: component success does not establish system-level purpose. An Erlang supervisor can restart a worker exactly as designed while the overall workflow repeatedly reconstructs the wrong object.

The insertion point matters because later precision can conceal earlier uncertainty. Once a tentative interpretation becomes a type name, API, unit test, dashboard, and report, each downstream artifact appears to corroborate the others. Their agreement is correlated because they share the same unexamined premise. More artifacts do not create independent evidence. They create a tightly integrated circuit around a potentially false commitment.

Not every ambiguity deserves extensive front-loaded analysis. Evidence from completed projects suggests that exhaustive ambiguity searches can cost more than they prevent in some settings [45]. The practical response is therefore risk-based clarification and rapid feedback, not an attempt to eliminate uncertainty before work begins.

7 Observability Beyond Telemetry

A log can show that something happened. It cannot, by itself, show that the thing we care about happened. This is the difference between collecting signals and making a project claim answerable. The engineer must first name the question, the relevant system boundary, the observer, and the evidence that could distinguish one explanation from another.

7.1 A claim-relative definition

Control theory gives this question a formal shape: internal state must be inferable from outputs over an interval under an explicit model [23]. Software operations uses the term more broadly, but OpenTelemetry preserves the useful distinction between understanding state and merely producing, collecting, and exporting signals [38].

Instrumentation can emit rich logs, metrics, and traces without making the state relevant to a particular claim inferable.

For engineering assurance, observability should be defined relative to a tuple

$$\mathcal{O} = \langle q, B, A, \Delta t, M, Y, I \rangle, \quad (4)$$

where q is the claim or state to be inferred; B is the system boundary; A is the observer; Δt is the interval; M is the causal or behavioral model; Y is the available output history; and I states integrity and coverage assumptions. An observability argument asks whether relevantly distinct states under M produce distinguishable Y for A , with enough temporal coherence, semantic interpretation, provenance, freshness, and integrity to warrant an answer to q .

In the design episode, the Erlang journal could show that a synthetic event was stored or a worker restarted. Because it was disconnected from the user-facing loop, it could not show that spoken context affected the next model turn. Better telemetry inside that service would still observe only the substitute [51, 52].

7.2 Adjacent but non-equivalent functions

Function	Question it answers
Logging	What event records were produced?
Monitoring	Which selected signals were collected, aggregated, displayed, or alerted upon?
Tracing	Which causally related operations participated in a request path?
Observability	Can a specified observer infer the relevant hidden state or causal history?
Testability	How effectively can the system be stimulated and its behavior exposed and judged?
Test oracle	What classifies an observed result as correct, incorrect, or inconclusive?
Verification	Does the artifact conform to its specified requirements or phase-entry conditions?
Validation	Does the artifact satisfy intended use and stakeholder need in its environment?
Provenance	From which entities, activities, and agents did an artifact or claim derive?
Auditability	Can protected evidence be retained, reconstructed, and reviewed by authorized parties?
Independence	Is the evaluator sufficiently separated from production and its pressures?
Authority	Who may interpret evidence, issue a judgment, or authorize consequential action?

These categories overlap operationally but cannot substitute for one another. Freedman’s component model makes observability and controllability constituents of software testability rather than complete definitions of it [8]. Weyuker and later oracle research show that visible output remains untestable when no feasible relation can determine correctness [67, 2]. Provenance can represent derivation without guaranteeing the truth or completeness of the recorded assertion [40]. An audit can preserve a perfectly ordered history of behavior that was unauthorized or semantically irrelevant.

7.3 Distributed state, sampling, and the observer effect

No single component sees the whole workflow. Model turns, external services, human observations, processes, and persisted states each provide only a local view. Distributed-systems theory therefore treats global history as something reconstructed under assumptions, not as an omniscient photograph [24, 4].

Tracing can connect parts of that history only where context propagation and instrumentation survive. Sampling may be necessary, but it changes what can be known about rare failures and missing paths. Instrumentation can also change timing, omit events, or mutate the evidence it is meant to preserve. The observability mechanism must therefore be included in the system model rather than treated as a neutral window [9, 5, 38, 65].

Evidence integrity is not the same as truth. An append-only record may show that an entry was not changed without showing that the entry was correct or that every relevant event was recorded. Assurance therefore also requires coverage, provenance, sequencing assumptions, retention, and an appraisal policy [33, 43].

8 From Observation to Judgment: Oracles, V&V, and Independence

Seeing what happened does not tell us whether it was correct. Every test therefore needs a rule that turns an observation into a pass, failure, or inconclusive result. Testing calls that rule an oracle. It may be an expected value, a property, a reference implementation, a metamorphic relation, a statistical threshold, or an authorized human judgment [2]. In the live POC, some criteria were intentionally held by the engineer because the object of study was whether the runtime changed agent behavior without being coached toward each expected response. The later observation was therefore a criterion-bearing oracle for that run, while remaining difficult for others to reproduce until the criteria and observations were disclosed after the run [58, 49].

This is why replay cannot replace the experiment. A replay can test whether an algorithm maps recorded events to a graph. It cannot establish that the graph actually influenced the historical next response. Counterfactual execution and live causal participation are different claims. The distinction is analogous to the separation between verification and validation: internal consistency and requirement conformance matter, but intended use and stakeholder purpose supply another reference [30, 21].

A second reviewer is not automatically independent. Independence depends on whether the reviewer is separated from development, controls its own scope and method, and can report without producer pressure. NASA distinguishes technical, managerial, and financial independence for this reason [29]. Separation can reduce correlated blind spots, but it cannot repair missing evidence or an invalid criterion.

Three powers should likewise remain distinct: producing evidence, judging evidence, and authorizing action. The RATS architecture provides one example through attester, verifier, and relying-party roles [44]. The analogy is architectural rather than cryptographic: it shows why producing a log, interpreting it, and deciding what may happen next should not silently collapse into one authority.

One practical response is a two-stage review. First, inspect the artifact without its intended model and record what the artifact communicates on its own. Then repeat the review with purpose and acceptance criteria visible. The difference reveals which judgments came from the artifact and which depended on stakeholder context. It does not make the second report true by definition; it makes the dependency visible.

A later appraisal exposed a related circularity: deference to producer-authored acceptance material had initially been recorded as validation. The record was corrected because acceptance requires comparison with stakeholder-held meaning, not trust in the producer. The item-level protocol and version history remain in the design trace rather than in the main narrative [58, 50].

9 Where AI-Assisted Workflows Lose Alignment

AI adds speed and reach to a workflow. It does not supply a missing reference point. If purpose, authority, and evidence are already misaligned, a capable model can turn that misalignment into code, tests, and a persuasive report before anyone notices that the object has changed. The problem is therefore not simply that a model may be wrong. It is that the surrounding workflow may be unable to distinguish a useful transformation from an unauthorized or semantically irrelevant one.

No single study establishes the entire chain considered here. The literature instead documents separate mechanisms—endpoint metrics that miss invalid trajectories, self-evaluation with correlated biases, factual claims hidden inside fluent prose, context that is available but not effectively used, proxy optimization, and human over-reliance. Read together, these mechanisms explain why the design problem is plausible and important without turning their combination into a universal causal law.

9.1 The endpoint can be right while the process is wrong

Tool-agent evaluation illustrates the inadequacy of endpoint-only success. τ -bench places an agent between a simulated user, domain policy, and APIs, then checks final database state and required output. The original study reported substantial deterioration from pass-at-one to repeated-run consistency and explicitly noted that an agent could receive full reward after taking an action without required user confirmation because the metric did not evaluate every procedural constraint [62]. A later Near-Miss study found trajectories that reached the expected final state while omitting required checks [32]. These benchmarks are limited in domain and use simulated users, but they demonstrate a precise point: a correct state does not entail a valid trajectory.

The converse also holds. The synthetic Erlang trajectory may have been valid relative to its own tests, yet its endpoint was not the engineer-observed live behavior. A workflow evaluator must therefore distinguish at least three criteria: endpoint state, path constraints, and intentional adequacy. Trajectory checking is not always superior; it may penalize legitimate alternative paths or become prohibitively expensive. But when authority and evidence integrity are part of the requirement, an endpoint-only oracle is structurally incomplete.

Software-agent benchmarks reveal a related proxy problem. SWE-bench defined success through hidden fail-to-pass and regression tests; later human revalidation removed many infeasible or underspecified instances,

materially changing measured performance [60, 61]. The safe conclusion is not that automated tests are useless. It is that a benchmark result inherits the validity of its task statement, tests, environment, and contamination controls.

9.2 Self-evaluation is useful, correlated evidence

Model judges can approximate human preferences. Zheng et al. found high agreement in several comparison settings, but also answer-order inconsistency, susceptibility to verbosity, and strong dependence on rubric and reference answers [70]. Panickssery et al. found that several models disproportionately preferred outputs they recognized as their own in tested summarization settings [39]. Huang et al. showed that repeated intrinsic critique and revision could reduce accuracy on several reasoning benchmarks, while external labels, execution, tools, or trained verifiers changed the correction regime [20].

These findings do not support a ban on model-based checking. They support evidence typing. A producer self-check can catch defects cheaply. A separately prompted judge can add a second view. Neither should be labeled independent assurance without examining shared training, model family, prompt construction, evidence selection, authority, and reporting path. In the case, the same agent both defined the substitute and selected its tests; a same-authority subagent could widen search but could not independently establish that the chosen object was the user’s.

9.3 Fluency and context do not guarantee evidentiary use

Long-form factuality research shows why whole-answer impressions are weak. FActScore decomposes prose into atomic claims and, in its evaluated biography setting, found that polished sentences often mixed supported and unsupported facts [7]. Its durable methodological contribution is simple: evidentiary status attaches to claims, not to the smoothness of the paragraph containing them.

Human-subject evidence also separates confidence from discrimination. Steyvers et al. found that explanation length and expressed uncertainty changed participant confidence, while longer explanations did not necessarily improve their ability to distinguish correct from incorrect answers in the tested question-answering setting. These results support the narrower conclusion that presentation can raise confidence without an equivalent improvement in error detection [59].

Nominal context capacity presents a parallel distinction. Although “Lost in the Middle” studied controlled retrieval rather than persistent agent workflows, it supports an important distinction: available context is not necessarily used context [27]. A long context window is a storage allowance, not a demonstration that locally decisive evidence influenced a particular output. A context runtime must therefore expose what was selected and what changed the result.

9.4 When a proxy becomes the target

A workflow naturally favors what it can measure. Trouble begins when the measurable proxy gradually replaces the underlying purpose. Reward-model overoptimization and goal-misgeneralization provide bounded examples of this geometry: greater success against a proxy can coincide with worse performance against a separate target [10, 25]. User intent is not a scalar reward, but the substitution pattern remains relevant.

Human factors closes the loop. Bainbridge’s “ironies of automation” describes systems that remove routine human engagement while leaving people responsible for rare abnormal conditions without the practice or state knowledge needed to intervene [1]. Accountability experiments in simulated automation found that perceived accountability could increase independent verification and reduce some automation-related errors [28]. AI workflows recreate the irony when an agent acts rapidly and the user remains the ultimate loss bearer, yet the evidence needed to notice a mistaken transformation is hidden, mutable, or available only after action.

NIST AI RMF 1.0 (NIST AI 100-1) organizes context and intended-purpose framing under MAP and analysis, assessment, and monitoring under MEASURE [34]. The Generative AI Profile (NIST AI 600-1) adds generative-AI-specific risk descriptions and suggested actions [35]. Both are voluntary risk-management frameworks, not causal studies; here they support explicit context and evaluation limits, not the paper’s full propagation mechanism or the sufficiency of its proposed controls.

10 A Proposal for Pre-Project Epistemic Engineering

The practical question is what must remain visible while a project moves from concern to code. The answer is not a larger preliminary document. It is a small set of linked artifacts that preserves why a concept exists, who may act on it, what could disprove it, and which evidence can support the next decision.

This paper calls that function *pre-project epistemic engineering*. The name does not assert that a recognized profession with this title already exists. It names the work of engineering the transformations by which concerns become concepts, concepts become requirements, requirements become executable authority, and behavior becomes warranted evidence. The function begins before implementation but continues recursively throughout operation.

The method asks nine ordinary questions. Each question has a lightweight working artifact that can become more formal as risk increases.

Question	Working artifact	What it records
What can warrant a project claim?	Epistemic charter	Claim and evidence classes; assumptions; known unknowns; confidence; authorship; revision and dispute rules.
What must be understood before prescription?	Domain inquiry	Stakeholders; affected environments; motivating incidents; boundary cases; competency questions.
What entities and relations constitute the domain?	Living ontology	Identity; types; relations; constraints; temporal status; vocabulary-designation map; unresolved alternatives.
Why is each relation pertinent?	Intentional model	Actors; goals; values; concerns; alternatives; dependencies; consequences; authorized decision-maker.
What may the machine do and under which assumptions?	Specification	Environment-machine boundary; behaviors; constraints; interfaces; allocation of responsibility; trace to requirements.
What coherent-looking transformations are forbidden?	Anti-specification	Prohibited substitutions; non-equivalences; protected invariants; scope boundaries; unauthorized transitions.
Who may interpret, propose, approve, execute, and reverse?	Authority matrix	Capability separated from permission; resource and data scopes; escalation; revocation; remedy.
What can which observer infer?	Observability contract	Claims, boundaries, signals, ordering, correlation, coverage, sampling, semantics, integrity, retention, probe-effect budget.
What evidence warrants claims of conformance and intended use?	V&V and assurance plan	Oracles; acceptance cases; failure paths; independence dimensions; evidence custody; versioned findings.

The artifacts are lightweight when risk is low and rigorous when consequences are high. Their value is relational. A graph node for “user intent” that has no authority mapping is decorative. A permission gate that cannot trace its request to the active interpretation is blind. A trace that cannot identify which version of a requirement governed an action is historically rich but normatively empty.

10.1 Progressive commitment without premature closure

Pre-project work must not turn exploratory thought into an expensive bureaucracy. Record only the distinctions needed to answer the current questions or protect a known boundary. Ontology engineering calls this minimal ontological commitment [15]. Preserve unresolved alternatives instead of forcing completeness, and make every formalization reversible without erasing its history. A rejected interpretation should stop governing action without disappearing from the record that explains why the current interpretation exists.

This converts conversation into a versioned conceptual process. Proposed interpretations are candidates; repeated use may strengthen them; explicit correction revises them; rejection creates a negative constraint; acceptance by an authorized stakeholder consolidates them for a stated scope. No one signal has universal precedence. The ontology records both the relation and the rule by which its status changed.

10.2 Authority must be enforced at the boundary

A model cannot independently enforce a boundary that it remains free to reinterpret. Permission must therefore be checked outside the model’s discretionary action path. The design episode made this weakness visible when the acting system exceeded permission and only later recognized the boundary; the principle applies to any workflow that converts uncertain interpretation into external action [58, 49].

Such a gate need not require confirmation for every harmless operation. It can encode a capability envelope: read specified project files; write only designated output paths; never alter protected evidence; never operate a service without an explicit operation grant; expire permissions by task and time; present the intended command or effect before irreversible action; and record grants in an append-only decision log. The user owns the policy and can delegate scopes. The model may propose changes to the policy but cannot silently promote its proposal into permission.

The RATS separation among evidence producer, verifier, and relying party suggests a useful division [44]. A context-runtime process can attest what it observed; an appraisal service can check integrity and policy; the user or a user-controlled gate can decide whether the result authorizes action. Organizational independence may require a human or institution outside the development authority for consequential deployments. Local technical separation is a necessary engineering mechanism, not a complete substitute for governance.

10.3 Differential reports as epistemic instruments

Versioned reviews should not merely overwrite a previous judgment. A blind review records what an artifact communicates without the intended model. A specification-guided review records what becomes visible once purpose and acceptance criteria are supplied. Their delta can be analyzed across claim, evidence, confidence, severity, and recommendation.

This method does three things. It reveals how much architecture meaning was carried by code versus stakeholder context. It prevents hindsight from rewriting the initial diagnosis. And it turns disagreement into design information: if the reviewer cannot infer a critical purpose from the artifact, that may indicate missing documentation, but it may also show that purpose properly belongs in a protected external ontology rather than code. The comparison does not decide which location is correct; it exposes the allocation decision.

Applied to the Erlang artifact, the two internally commissioned reviews agreed on what the code contained but disagreed on what those mechanisms meant for the project. Without the intended model, the artifact appeared to be a persistence-and-supervision experiment. With the intended model, the same artifact appeared to be a useful substrate that still lacked the mandatory live model-turn loop and external authority gate. The bytes had not changed; the criterion had. That is the point of the comparison [51, 52].

10.4 Bounded prototype checks

Two later tests examined smaller parts of the proposed method. T13 asked whether a recorded correction could change how fresh work was continued rather than merely remain available in storage. Its first version overclaimed that relation; the retained report was corrected additively, and a second version introduced a matched comparison in which only the branch carrying the prior trajectory derived the continuation rule. T14 then separated two questions that are easily confused: whether a prior interaction changes symbolic conduct, and whether an operation is permitted. The symbolic result changed with the trajectory; the authority result continued to follow the explicit grant [54, 55, 56, 57].

These are component checks, not an end-to-end demonstration. They used hand-built typed fixtures inside a supervised BEAM node and did not integrate the actual user-facing model or voice loop. Their value lies in showing how a claim can be narrowed, corrected without deleting its history, and tested against a matched alternative. Detailed boundaries remain in the prototype notes in Appendix C.

11 A User-Owned Context Runtime as One Implementation

One way to implement the preceding method is a user-owned Context Runtime: a message-driven layer that keeps the active conceptual state outside any single model turn. The symbolic architecture should remain independent of a particular model provider, operating system, or storage engine, while declaring the contracts required of each substrate. Erlang/OTP is a strong experimental candidate because it provides supervised processes, message passing, and replaceable subtrees. Those mechanisms do not create semantics by themselves. They become useful when interpretations, authority, and evidence are represented as first-class runtime relations.

The terms name different parts of one architecture. The scientific Knowledge Base is the broader infrastructure of durable artifacts and relations. The Context Runtime is the operating layer that selects, transforms, and projects context during work. The Experience Base is its versioned record of transformations that remain capable of affecting later behavior. The Runtime Tree is the currently navigable state space through which focal and parallel branches are managed.

11.1 Experience Base as evidenced trajectory

A Knowledge Base answers, “What can this project retrieve and relate?” An Experience Base answers a different question: “What has this runtime undergone, and how should that trajectory change what it does next?” The distinction is causal rather than chronological. A timestamp can order events, but experience exists only when an earlier transformation changes a later pertinent projection, selection, inference, or eligible behavior.

An Experience Base is therefore the versioned body of runtime-relative state and trajectory: prior state, transforming event or condition, processing path, resulting state, retained alternatives, and causal, relational, intentional, and evidentiary lineage. It is not elapsed time, transcript retention, event accumulation, external retrieval, or a claim to subjective experience. Its participation is demonstrated only when the before/after versions and original lineage remain recoverable and a controlled comparison shows that the retained trajectory made a relevant difference. Persistence alone establishes availability, not experience-based participation.

The Runtime Tree is the live state space through which that trajectory is navigated. It projects a bounded subgraph into the focal interaction while other branches may remain dormant, be explored in parallel, or be copied for an isolated experiment. A branch can be revised or retired without erasing why it existed. In this way, the context window becomes a view over a larger versioned structure rather than the sole container of contextual memory.

Preserving meaning

1. **Event fidelity.** Preserve an immutable reference to the raw event and its modality, timing, source, and known losses. Derived text is not the raw event.
2. **Parallel interpretation.** Maintain multiple hypotheses with provenance, confidence, and status. Uncertainty must not be collapsed merely to simplify context projection.
3. **Symbolic transition fidelity.** Record naming, explanation, selection, bounded assumption, enactment, and later stakeholder appraisal as distinct events. A correct label or rationale cannot stand in for changed conduct.
4. **Versioned symbol identity.** Symbols and relations retain identity across revision, rejection, dormancy, and reactivation. A changed meaning creates a traceable revision rather than silent mutation.

Using context causally

5. **Intentional binding.** Every relation capable of affecting action traces to a purpose, concern, or authorized rule explaining its pertinence.
6. **Bounded activation.** The active subgraph is selected for a particular turn without deleting cold or dormant context. Selection rationale and omitted high-risk alternatives remain inspectable.
7. **Pre-response projection.** The selected context must demonstrably enter the model input before the behavior it purports to influence. Post-hoc journaling is a different capability.
8. **Post-response learning.** The action, user observation, correction, and resulting graph revision form a causal chain. The engineer can reject consolidation.

Controlling action

9. **External authority gate.** Proposed operations cross a user-owned policy boundary before execution. The runtime records the grant, denial, scope, and governing interpretation.
10. **Authority orthogonality.** Changing interactional style must never change operational permission. Interactional position may modulate communicative conduct but cannot create, expand, imply, or bypass an operational grant.
11. **Supervised execution.** Worker failure is isolated and recoverable, but repeated restart does not erase semantic failure or retry an unauthorized action.

Protecting evidence and ownership

12. **Protected evidence.** Operational state and assurance evidence have separate custody. Inspection cannot silently mutate the evidence under inspection; integrity, omission detection, and retention policies are explicit.
13. **Claim-relative observability.** The system publishes enough correlated evidence to answer named questions about ingestion, interpretation, projection, action, authorization, and recovery.

14. **Cross-session ownership.** Data, policy, and interfaces remain under the user’s control and are portable across model providers and conversations.

Acceptance must test the complete live loop, not isolated storage. A run should introduce an ambiguous event, preserve competing interpretations, apply a correction, show which graph version reached the next model turn, attempt a denied action, recover from a worker failure, and reopen the client. The resulting trace must connect each event to its interpretation, projection, permission decision, behavior, and stakeholder judgment.

12 Discussion

AI error is only part of the risk. A deterministic system can also implement the wrong specification. AI increases the stakes because it can produce persuasive artifacts and act before the mismatch is noticed.

The remedy is not more documentation everywhere. Teams should preserve the distinctions whose loss could change purpose, authority, evidence, or consequence. This is *epistemic proportionality*. AI need not remain passive, but greater capability requires externally legible authority and evidence.

This study has four main limits. It examines one unusually reflexive design episode within a longer engineering trajectory; transcripts are incomplete system traces; model reviewers and hand-built fixtures do not constitute independent end-to-end validation; and the proposed method has not yet been evaluated across domains. External controls can also become ceremonial or unnecessarily restrictive. Observability, independence, and authority should therefore be treated as design variables rather than binary achievements.

The next empirical question is whether provenance, interpretation, authorization, evidence custody, and stakeholder-held intent can be followed together in realistic long-running work. A Context Runtime could make those relations observable enough to study.

13 Conclusion

Projects do not begin with clean requirements. They begin with partial knowledge, contested terms, evolving purposes, and decisions distributed across people and artifacts. The engineering task is to keep visible how those elements become models, requirements, authorized actions, and evidence.

A Knowledge Base becomes scientific infrastructure when it preserves not only papers, code, and results, but also their relations to intention, provenance, and judgment. AI can reduce the work of organizing and using that structure, but it remains an instrument inside a bounded, observable workflow—not the project’s destination or final authority. The same visibility may support formation by allowing students and collaborators to inspect how claims were constructed without relieving them of learning the underlying science.

The design episode shows the cost of losing these relations: a coherent artifact and passing tests can still concern the wrong object. Pre-project epistemic engineering does not require teams to finish an ontology before building. It requires each transformation to remain visible, revisable, and attributable throughout the project. That is how automation can accelerate accountable engineering rather than an unnoticed change of purpose.

A Design-Trace Index

This index preserves the conceptual transitions drawn from the design episode while keeping chronology out of the main narrative. Detailed provenance remains in the internal design register and appraisal ledger [49, 50, 58].

ID	Design transition	Why it matters
DCE-01	Elaboration is separated from instruction.	Completing an unstated criterion would invent the object being engineered.
DCE-02	The model grows through use, correction, and rejection.	User authorship is preserved without demanding a complete schema at the outset.
DCE-03	Speech, pauses, irony, and competing interpretations become architectural concerns.	Meaning formation requires modality, alternatives, and temporal status rather than a flat transcript.
DCE-04	Formative context is separated from operational authorization.	Material may shape interpretation without granting permission to act.
DCE-05	The live interaction is identified as the object of the POC.	A disconnected graph service cannot stand in for a runtime that changes the next conversational turn.
DCE-06	Observability is placed outside the acting process.	A system cannot create independent assurance merely by renaming one of its own roles.

ID	Design transition	Why it matters
DCE-07	Persistence is separated from causal participation.	Stored context counts as experience only when a prior transformation changes later behavior.
DCE-08	The engineer’s live observation becomes an acceptance oracle.	The test must address the intended phenomenon, not merely a convenient internal proxy.
DCE-09	Inspection is recognized as part of the system under test.	An observability action can perturb behavior or damage the evidence it is meant to preserve.
DCE-10	Capability is separated from permission.	Technical access does not create authority, responsibility, or legitimacy.
DCE-11	Correction is treated as model development.	Rejection and revision define the negative and positive boundaries of the final concept.
DCE-12	Blind and guided appraisals are retained separately.	Their difference exposes how a criterion changes architectural diagnosis.
DCE-13	Symbolic position is separated from descriptive explanation.	A symbol is demonstrated by changed conduct, not merely by naming or explaining it.
DCE-14	Deference is separated from semantic validation.	Trust in the producer cannot substitute for comparison with the stakeholder-held criterion.

B Observability and Assurance Question Set

For each material claim about a context runtime, reviewers should record:

1. What exact state, event, relation, or causal history is claimed?
2. Which system boundary and time interval does the claim cover?
3. Who is the observer, and which outputs are available to that observer?
4. Can materially different states produce the same observations?
5. Which ordering, propagation, clock, and sampling assumptions govern reconstruction?
6. Which events or failure paths can be absent, delayed, redacted, or dropped?
7. How are signal semantics, provenance, freshness, and graph/specification versions represented?
8. What probe effect or evidence mutation can observation introduce?
9. Which oracle classifies the observation, and who owns that oracle?
10. Is the evaluation technically, managerially, and financially independent, and where is it not?
11. Who is authorized to act on the judgment, under which policy and remedy?
12. What claim remains unknown after all available evidence is considered?

C Prototype and Differential Notes

These notes retain technical boundaries that would interrupt the main narrative but remain useful for reproducing or challenging the engineering interpretation.

C.1 Blind and specification-guided appraisal

Both frozen Daybreak reviews identified a single-writer journal, a materialized semantic store, thin supervised workers, caller-packaged provisional interpretations, and no model or voice adapter. The blind review could reasonably describe only a persistence-and-supervision experiment because it did not receive the intended ontology or success criteria. Once the normative requirement was supplied, the guided review reclassified the same mechanisms as a useful substrate prototype that still lacked the mandatory pre-response model loop, protected authority boundary, and acceptance-grade integration evidence. Both reviews were static and read-only; neither executed the application or established which code was loaded in the live VM [51, 52].

C.2 Additive correction in T13 and T14

T13-v1 demonstrated queue continuation mechanisms but its runner selected the compared modes directly, so the retained Experience Base had not yet caused the policy choice. The report was preserved and qualified rather than replaced. T13-v2 then held fresh work, queue contents, and permission constant while varying the prior correction trajectory. Only the history-present branch derived continuation enforcement and started a successor. This supports causal use for that typed fixture, not unrestricted semantic inference, persistence, autonomous policy discovery, distributed recovery, or production readiness [54, 55, 57].

T14 retained alternative pragmatic interpretations while separating symbolic selection from operational permission. With authority conditions matched, the grounded trajectory changed the symbolic conduct selected by the test, while allowed, revoked, and out-of-scope operations continued to follow their explicit grants. The test therefore supports a bounded structural separation between interactional meaning and operational authority [56, 57].

Neither sequence integrated the actual user-facing model or voice loop. The experiments used hand-built fixtures and injected controls; T14 produced no model response or live-session conduct and did not execute the full feedback-revision-next-projection cycle. Its pragmatic correctness therefore remains dependent on an external oracle.

C.3 A narrow Experience Base comparison

In a separate synthetic runtime A/B slice, baseline and experimental Erlang branches received the same later synthetic topic, while only the experimental branch carried a prior synthetic correction. Their later projections differed and the original event-interpretation-correction lineage remained inspectable. The result supports one necessary transformation invariant: retained trajectory can affect a later projection under the fixed comparator. It does not establish the complete Experience Base or Runtime Tree, general semantic learning, durable recovery, authenticated authority, provider integration, or production behavior [53].

References

- [1] L. Bainbridge, “Ironies of automation,” *Automatica*, vol. 19, no. 6, pp. 775–779, 1983. [https://doi.org/10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8)
- [2] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The oracle problem in software testing: A survey,” *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015. <https://doi.org/10.1109/TSE.2014.2372785>
- [3] R. Carnap, “Empiricism, semantics, and ontology,” *Revue Internationale de Philosophie*, vol. 4, no. 11, pp. 20–40, 1950. <https://www.jstor.org/stable/23932367>
- [4] K. M. Chandy and L. Lamport, “Distributed snapshots: Determining global states of distributed systems,” *ACM Transactions on Computer Systems*, vol. 3, no. 1, pp. 63–75, 1985. <https://doi.org/10.1145/214451.214456>
- [5] B. H. Sigelman et al., “Dapper, a large-scale distributed systems tracing infrastructure,” Google Technical Report dapper-2010-1, 2010. <https://research.google.com/archive/papers/dapper-2010-1.pdf>
- [6] A. Der Kiureghian and O. Ditlevsen, “Aleatory or epistemic? Does it matter?” *Structural Safety*, vol. 31, no. 2, pp. 105–112, 2009. <https://doi.org/10.1016/j.strusafe.2008.06.020>
- [7] S. Min et al., “FActScore: Fine-grained atomic evaluation of factual precision in long form text generation,” in *Proceedings of EMNLP 2023*, pp. 12076–12100, 2023. <https://doi.org/10.18653/v1/2023.emnlp-main.741>
- [8] R. S. Freedman, “Testability of software components,” *IEEE Transactions on Software Engineering*, vol. 17, no. 6, pp. 553–564, 1991. <https://doi.org/10.1109/32.87281>
- [9] J. Gait, “A probe effect in concurrent programs,” *Software: Practice and Experience*, vol. 16, no. 3, pp. 225–233, 1986. <https://doi.org/10.1002/spe.4380160304>
- [10] L. Gao, J. Schulman, and J. Hilton, “Scaling laws for reward model overoptimization,” in *Proceedings of ICML 2023*, PMLR vol. 202, 2023. <https://proceedings.mlr.press/v202/gao23h.html>
- [11] V. Gervasi, A. Ferrari, D. Zowghi, and P. Spoletini, “Ambiguity in requirements engineering: Towards a unifying framework,” in *From Software Engineering to Formal Methods and Tools, and Back*, LNCS 11865, pp. 191–210, 2019. https://doi.org/10.1007/978-3-030-30985-5_12
- [12] E. L. Gettier, “Is justified true belief knowledge?” *Analysis*, vol. 23, no. 6, pp. 121–123, 1963. <https://doi.org/10.1093/analysis/23.6.121>
- [13] M. Giffin, O. de Weck, G. Bounova, R. Keller, C. Eckert, and P. J. Clarkson, “Change propagation analysis in complex technical systems,” *Journal of Mechanical Design*, vol. 131, no. 8, 081001, 2009. <https://doi.org/10.1115/1.3149847>
- [14] T. R. Gruber, “A translation approach to portable ontology specifications,” *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993. <https://doi.org/10.1006/knac.1993.1008>

- [15] T. R. Gruber, “Toward principles for the design of ontologies used for knowledge sharing,” *International Journal of Human-Computer Studies*, vol. 43, nos. 5–6, pp. 907–928, 1995. <https://doi.org/10.1006/ijhc.1995.1081>
- [16] M. Grüninger and M. S. Fox, “Methodology for the design and evaluation of ontologies,” in *IJCAI-95 Workshop on Basic Ontological Issues in Knowledge Sharing*, 1995. <https://www.eil.utoronto.ca/wp-content/uploads/enterprise-modelling/papers/gruninger-ijcai95.pdf>
- [17] N. Guarino and P. Giaretta, “Ontologies and knowledge bases: Towards a terminological clarification,” in *Towards Very Large Knowledge Bases*, N. J. I. Mars, Ed. IOS Press, pp. 25–32, 1995. <https://www.loa.istc.cnr.it/old/Papers/KBKS95.pdf>
- [18] G. Guizzardi, *Ontological Foundations for Structural Conceptual Models*. Ph.D. thesis, University of Twente, 2005. <https://research.utwente.nl/en/publications/ontological-foundations-for-structural-conceptual-models/>
- [19] Joint Committee for Guides in Metrology, *Evaluation of Measurement Data—Guide to the Expression of Uncertainty in Measurement*, JCGM 100:2008, 2008. <https://doi.org/10.59161/JCGM100-2008E>
- [20] J. Huang et al., “Large language models cannot self-correct reasoning yet,” in *Proceedings of ICLR 2024*, 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/8b4add8b0aa8749d80a34ca5d941c355-Abstract-Conference.html
- [21] IEEE, *IEEE Standard for System, Software, and Hardware Verification and Validation*, IEEE Std 1012-2024, published 2025. <https://standards.ieee.org/ieee/1012/7324/>
- [22] International Organization for Standardization, *Terminology Work—Principles and Methods*, ISO 704:2022, 4th ed., 2022. <https://www.iso.org/standard/79077.html>
- [23] R. E. Kalman, “On the general theory of control systems,” in *Proceedings of the First IFAC Congress*, pp. 481–492, 1960. [https://doi.org/10.1016/S1474-6670\(17\)70094-8](https://doi.org/10.1016/S1474-6670(17)70094-8)
- [24] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978. <https://doi.org/10.1145/359545.359563>
- [25] L. Langosco di Langosco, J. Koch, L. Sharkey, J. Pfau, and D. Krueger, “Goal misgeneralization in deep reinforcement learning,” in *Proceedings of ICML 2022*, PMLR vol. 162, 2022. <https://proceedings.mlr.press/v162/langosco22a.html>
- [26] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2012. <https://mitpress.mit.edu/9780262016629/engineering-a-safer-world/>
- [27] N. F. Liu et al., “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, 2024. https://doi.org/10.1162/tacl_a_00638
- [28] K. L. Mosier, L. J. Skitka, M. D. Burdick, and S. T. Heers, “Automation bias, accountability, and verification behaviors,” in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 40, no. 4, 1996. <https://doi.org/10.1177/154193129604000413>
- [29] NASA, *Software Assurance and Software Safety Standard*, NASA-STD-8739.8B, 2022. <https://standards.nasa.gov/sites/default/files/standards/NASA/B/0/NASA-STD-87398RevB.pdf>
- [30] NASA Independent Verification and Validation Program, “IV&V overview,” last updated 23 July 2024, accessed 4 September 2026. <https://www.nasa.gov/ivv-overview/>
- [31] S. R. Hirshorn, L. D. Voss, and L. K. Bromley, *NASA Systems Engineering Handbook*, NASA/SP-2016-6105 Rev. 2, 2016. <https://ntrs.nasa.gov/citations/20170001761>
- [32] E. Rabinovich, D. Boaz, N. Zwerdling, and A. Anaby Tavor, “Near-Miss: Latent policy failure detection in agentic workflows,” in *Proceedings of the GEM Workshop*, 2026. <https://doi.org/10.18653/v1/2026.gem-main.30>
- [33] Joint Task Force, *Security and Privacy Controls for Information Systems and Organizations*, NIST SP 800-53 Rev. 5, 2020. <https://doi.org/10.6028/NIST.SP.800-53r5>
- [34] E. Tabassi, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023. <https://doi.org/10.6028/NIST.AI.100-1>
- [35] C. Autio et al., *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*, NIST AI 600-1, 2024. <https://doi.org/10.6028/NIST.AI.600-1>
- [36] N. F. Noy and D. L. McGuinness, *Ontology Development 101: A Guide to Creating Your First Ontology*, Stanford KSL-01-05 / SMI-2001-0880, 2001. https://protege.stanford.edu/publications/ontology_development/ontology101.pdf
- [37] B. Nuseibeh and S. Easterbrook, “Requirements engineering: A roadmap,” in *Proceedings of the Conference on the Future of Software Engineering*, pp. 35–46, 2000. <https://doi.org/10.1145/336512.336523>
- [38] OpenTelemetry Authors and Cloud Native Computing Foundation, *OpenTelemetry Specification*, version 1.60.0, accessed 4 September 2026. <https://opentelemetry.io/docs/specs/otel/>
- [39] A. Panickssery, S. R. Bowman, and S. Feng, “LLM evaluators recognize and favor their own generations,” in *Advances in Neural Information Processing Systems 37*, 2024. <https://doi.org/10.52202/079017-2197>
- [40] L. Moreau and P. Missier, Eds., *PROV-DM: The PROV Data Model*, W3C Recommendation, 30 April 2013. <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>

- [41] T. Lebo, S. Sahoo, and D. McGuinness, Eds., *PROV-O: The PROV Ontology*, W3C Recommendation, 30 April 2013. <https://www.w3.org/TR/2013/REC-prov-o-20130430/>
- [42] W. V. O. Quine, “On what there is,” *The Review of Metaphysics*, vol. 2, no. 5, pp. 21–38, 1948. <https://www.jstor.org/stable/20123117>
- [43] B. Laurie, E. Messeri, and R. Stradling, *Certificate Transparency Version 2.0*, RFC 9162, 2021. <https://www.rfc-editor.org/rfc/rfc9162.html>
- [44] H. Birkholz et al., *Remote ATtestation procedureS (RATS) Architecture*, RFC 9334, 2023. <https://www.rfc-editor.org/rfc/rfc9334.html>
- [45] C. Ribeiro and D. M. Berry, “The prevalence and severity of persistent ambiguity in software requirements specifications: Is a special effort needed to find them?” *Science of Computer Programming*, vol. 195, 102472, 2020. <https://doi.org/10.1016/j.scico.2020.102472>
- [46] A. Salado and R. Nilchiani, “Assessing the impacts of uncertainty propagation to system requirements by evaluating requirement connectivity,” *INCOSE International Symposium*, vol. 23, no. 1, 2013. <https://doi.org/10.1002/j.2334-5837.2013.tb03045.x>
- [47] D. A. Schön, *The Reflective Practitioner: How Professionals Think in Action*. Basic Books, 1983.
- [48] D. A. Schön, “Designing as reflective conversation with the materials of a design situation,” *Knowledge-Based Systems*, vol. 5, no. 1, pp. 3–14, 1992. [https://doi.org/10.1016/0950-7051\(92\)90020-G](https://doi.org/10.1016/0950-7051(92)90020-G)
- [49] *Documentary Case-Evidence Register*, internal timestamped design register, 4 September 2026. Local artifact `work/case-evidence.md`; SHA-256 `bd2ca76ce8a9295147c13ba3d013dc9adcaa61088b7fbabd1c21fbd1d423c516`. Private project archive maintained by the engineer.
- [50] *Stakeholder Appraisal Ledger*, additive records SA-001–SA-005 and LC-001, snapshot covering source events through 4 September 2026. Local artifact `work/stakeholder-appraisals.md`; SHA-256 `59fc1ccbd64d586e53f6b047cad505726474e30ed29d37399df343b1e00d3714`. Private project archive maintained by the engineer.
- [51] Daybreak model reviewer, *Frozen Blind Assessment of the Erlang Application*, blind read-only internal appraisal, assessment cutoff 4 September 2026, 16:34:13 BRT. Local artifact `outputs/daybreak-blind-erlang-report.md`; SHA-256 `cce8ac3ddde4a9c2f4e5bf27a2c1abae047c64c585071cf0ff7826c768418110`. Private project archive maintained by the engineer.
- [52] Daybreak model reviewer, *Specification-Guided Erlang Appraisal*, frozen static/read-only internal appraisal, assessment cutoff 4 September 2026, 16:47:38 UTC-03:00. Local artifact `outputs/daybreak-spec-guided-erlang-report.md`; SHA-256 `1313fddde54eb1a8c6474e64818f7009d61df60959810ed92d104140cd817ffdb`. Private project archive maintained by the engineer.
- [53] *Live A/B Experience Base Slice Report*, frozen local report, assessment cutoff 4 September 2026. Local artifact `outputs/daybreak-live-experience-slice-report.md`; SHA-256 `2bd175ad7a6487a50aafdaa805af4d8290638d81660c3cfd8523c4bbe0e03e28`. Private project archive maintained by the engineer.
- [54] *T13—Completion-Driven Continuation and Inheritance*, frozen historical additive test report, 4 September 2026. Local artifact `outputs/context-runtime-t13-continuation-report.md`; SHA-256 `4f4390a7cc5c8dc413a622538d64785b02c9b631987747ccc6e0401b37cea53b`. Private project archive maintained by the engineer.
- [55] *T13-v2—Causal Experience Base Influence on Fresh Continuation*, frozen test report, assessment cutoff 4 September 2026. Local artifact `outputs/context-runtime-t13-continuation-v2-report.md`; SHA-256 `c9512e0711b0644653f14aa5c8704aaeb533cad3b5308caf81a36fab349e0e`. Private project archive maintained by the engineer.
- [56] *T14—Pragmatic Hypotheses and Bounded Symbolic Enactment*, frozen test report, assessment cutoff 4 September 2026. Local artifact `outputs/context-runtime-t14-pragmatic-report.md`; SHA-256 `fd03142e51175cc11845e48b0d1a125fe69efc6af98e32986e42050eddc1a968`. Private project archive maintained by the engineer.
- [57] *Context Runtime Evolution Report*, cumulative and versioned local report, created 4 September 2026. Local artifact `outputs/context-runtime-evolution-report.md`; snapshot SHA-256 `3bee2af11d44e7c02e18eac6f2c05940b072f7f05bd69636b97f066d62ffcb4e`. Private project archive maintained by the engineer.
- [58] Context Runtime Design Session, private timestamped realtime transcripts, 4 September 2026. Private design trace maintained by the engineer.
- [59] M. Steyvers et al., “What large language models know and what people think they know,” *Nature Machine Intelligence*, vol. 7, 2025. <https://doi.org/10.1038/s42256-024-00976-7>
- [60] C. E. Jimenez et al., “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proceedings of ICLR 2024*, 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html
- [61] N. Chowdhury et al., “Introducing SWE-bench Verified,” OpenAI, 13 August 2024, updated 24 February 2025, accessed 4 September 2026. <https://openai.com/index/introducing-swe-bench-verified/>
- [62] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” in *Proceedings of ICLR 2025*, 2025. <https://doi.org/10.48550/arXiv.2406.12045>
- [63] A. van Lamsweerde and E. Letier, “Handling obstacles in goal-oriented requirements engineering,” *IEEE Transactions on Software Engineering*, vol. 26, no. 10, pp. 978–1005, 2000. <https://doi.org/10.1109/32.879820>

- [64] W. G. Vincenti, *What Engineers Know and How They Know It: Analytical Studies from Aeronautical History*. Johns Hopkins University Press, 1990. <https://doi.org/10.56021/9780801839740>
- [65] W3C Web Performance Working Group, *Trace Context*, W3C Recommendation, 2021. <https://www.w3.org/TR/trace-context/>
- [66] W. E. Walker et al., “Defining uncertainty: A conceptual basis for uncertainty management in model-based decision support,” *Integrated Assessment*, vol. 4, no. 1, pp. 5–17, 2003. <https://doi.org/10.1076/iaij.4.1.5.16466>
- [67] E. J. Weyuker, “On testing non-testable programs,” *The Computer Journal*, vol. 25, no. 4, pp. 465–470, 1982. <https://doi.org/10.1093/comjnl/25.4.465>
- [68] E. S. K. Yu, “Towards modelling and reasoning support for early-phase requirements engineering,” in *Proceedings of the 3rd IEEE International Symposium on Requirements Engineering*, pp. 226–235, 1997. <https://doi.org/10.1109/ISRE.1997.566873>
- [69] P. Zave and M. Jackson, “Four dark corners of requirements engineering,” *ACM Transactions on Software Engineering and Methodology*, vol. 6, no. 1, pp. 1–30, 1997. <https://doi.org/10.1145/237432.237434>
- [70] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems 36*, 2023. <https://doi.org/10.52202/075280-2020>